Hoja de ejercicios — Git local, modelo mental, Markdown y primeras ramas

Propósito de la hoja

En esta hoja vas a trabajar Git en dos fases. Primero usarás un entorno visual para entender mejor qué son los commits, las ramas, `HEAD` y los merges sencillos. Después pasarás al trabajo real en local con un repositorio propio, archivos Markdown, staging, Conventional Commits, `--amend`, `stash` y una primera rama de trabajo.

El objetivo no es producir muchos documentos de respuesta, sino aprender a trabajar con Git con criterio desde el principio.

Qué se espera de ti

En esta sesión debes centrarte en comprender:

- la diferencia entre workspace, index y repositorio local;
- cómo se construye un historial legible;
- por qué una rama sirve para aislar trabajo;
- cuándo tiene sentido usar `--amend` y `stash`.

---

1. Práctica visual guiada

Ejercicio 1. Primer contacto con Learn Git Branching

Accede a Learn Git Branching:

`https://learngitbranching.js.org/?locale=es_ES`

Completa una primera tanda de niveles introductorios centrados en estas ideas:

- crear commits;
- mover `HEAD`;
- crear ramas;
- cambiar de rama;

- observar un merge sencillo.

No necesitas anotar cada paso. Tu objetivo es mirar el grafo y entender qué representa.

Evidencia mínima

Crea un único archivo llamado `observaciones-lgb.md` y responde brevemente:

1. ¿Qué representa visualmente un commit?
2. ¿Qué relación ves entre `HEAD` y la rama activa?
3. ¿Qué cambia en el grafo cuando haces un merge?

> **Criterio de trabajo**
>
> No intentes “pasar pantallas” sin pensar. Lo importante aquí es que empieces a ver Git como una estructura de historial.

---

2. Creación del repositorio real

Ejercicio 2. Crear tu repositorio de práctica

Crea una carpeta llamada `git-sesion-02` y, dentro de ella, otra carpeta llamada `manual-git-local`.

Dentro de `manual-git-local`, crea estos archivos:

- `README.md`
- `notas.md`
- `glosario.md`
- `.gitignore`

Inicializa la carpeta como repositorio Git y comprueba su estado.

Objetivo del ejercicio

Debes comprobar que:

- los archivos existen en tu carpeta;
- Git detecta esos archivos;
- todavía no existe historial confirmado.

---

3. Primer trabajo con Markdown y `README.md`

Ejercicio 3. Redactar un `README.md` básico

Edita `README.md` para que incluya, como mínimo:

- un título principal;
- una breve descripción del repositorio;
- una lista con tres puntos;
- una sección llamada `Objetivo`;
- una línea donde aparezca el comando `git status`.

No uses HTML. Todo debe estar escrito en Markdown básico.

Objetivo del ejercicio

Debes entender que Git también versiona documentación y que `README.md` es una parte natural de un proyecto real.

---

4. Primeros commits con intención

Ejercicio 4. Primer commit obligatorio

Haz que el primer commit del repositorio incluya **solo** `README.md`.

Condiciones obligatorias

- No puedes usar `git add .`.
- Debes revisar el estado antes de hacer commit.
- El mensaje debe usar este prefijo obligatorio:

  docs:

Resultado esperado

Tu historial debe empezar con un commit documental claro y el resto de archivos deben seguir fuera del historial.

Ejercicio 5. Construir un historial mínimo legible

Ahora completa el repositorio con trabajo real sobre los otros archivos.

En `notas.md`

Escribe tres ideas breves sobre:

- workspace;
- staging;
- commit.

En `glosario.md`

Incluye definiciones breves de:

- repositorio;
- rama;
- merge.

Después registra el trabajo en **exactamente dos commits** siguiendo estas reglas:

1. un commit `docs:` para contenido explicativo;
2. un commit `chore:` para ajustes auxiliares del repositorio, incluyendo `.gitignore` si todavía no lo has hecho.

Restricciones

- No puedes mezclar todo en un único commit.
- Debes leer `git status` y `git diff` antes de cada commit.
- El historial debe poder leerse con claridad usando `git log --oneline`.

> **Criterio técnico**
>
> En esta sesión no se evalúa solo que “guardes cambios”, sino que sepas separar trabajo en unidades con sentido.

---

5. Comprobar qué hace cada comando

Ejercicio 6. Distinguir `status`, `diff` y `log`

Haz una modificación pequeña en `README.md` y otra en `notas.md`, pero no confirmes todavía nada.

Después utiliza:

- `git status`
- `git diff`
- `git log --oneline`

No tienes que entregar una respuesta larga. Basta con que seas capaz de explicar oralmente o por escrito, al final de la sesión, qué te muestra cada uno.

Comprobación que debes hacer

Asegúrate de entender:

- qué archivos han cambiado;
- qué contenido ha cambiado;
- qué forma parte ya del historial y qué no.

---

6. `git commit -am` y `git commit --amend`

Ejercicio 7. Probar el límite de `git commit -am`

Haz dos cosas:

1. modifica un archivo ya rastreado, por ejemplo `README.md`;
2. crea un archivo nuevo llamado `borrador.md`.

Después intenta usar `git commit -am`.

Objetivo del ejercicio

Debes descubrir por experiencia propia:

- qué tipo de archivos entra automáticamente;
- qué tipo de archivo queda fuera;
- por qué este comando no sustituye la revisión del estado del repositorio.

Ejercicio 8. Corregir el último commit con `--amend`

Haz un commit pequeño y, justo después, corrígelo usando `git commit --amend`.

Puedes hacerlo de una de estas formas:

- mejoras el mensaje del commit;

- añades un cambio pequeño que olvidaste incluir y que pertenece claramente al mismo trabajo.

Objetivo del ejercicio

Debes entender que `--amend` sirve para corregir el último commit del historial local cuando detectas un error inmediatamente.

---

7. `git stash` como aparcamiento temporal

Ejercicio 9. Guardar trabajo en curso sin hacer commit

Haz cambios incompletos en dos archivos ya rastreados. No los confirmes.

Después:

1. consulta el estado;
2. guarda temporalmente ese trabajo con `stash`;
3. comprueba qué ha ocurrido en el directorio de trabajo;
4. recupera después ese trabajo;
5. vuelve a comprobar el estado.

Objetivo del ejercicio

Debes distinguir claramente entre:

- trabajo en el workspace;
- trabajo confirmado en el historial;
- trabajo aparcado temporalmente en `stash`.

> ⚠ **Idea clave**
>
> `stash` no sustituye una buena organización del repositorio. Solo te ayuda a aparcar trabajo temporalmente.

---

8. Primeras ramas en local

Ejercicio 10. Crear una rama y trabajar en ella

Con el repositorio en estado limpio:

1. crea una rama llamada `docs/guia-inicial`;
2. cámbiate a ella con `git switch`;
3. añade una nueva sección a `README.md`;
4. haz un commit con prefijo `docs:`.

Después:

5. vuelve a la rama principal;
6. comprueba que ese último cambio no está ahí todavía.

Objetivo del ejercicio

Debes comprobar que una rama sirve para aislar trabajo y que crear una rama no significa duplicar carpetas ni crear otro proyecto distinto.

Ejercicio 11. Reconocer `git checkout`

Vuelve a cambiar de rama utilizando `git checkout` en lugar de `git switch`.

No necesitas redactar un documento aparte. Basta con que compruebes en la práctica que:

- `switch` te resulta más claro para cambiar de rama;
- `checkout` sigue siendo un comando que necesitas reconocer.

Ejercicio 12. Hacer un merge sencillo

Desde la rama principal, integra la rama `docs/guia-inicial`.

Después:

- revisa el historial;
- comprueba el contenido final del `README.md`.

Objetivo del ejercicio

Debes entender que:

- el merge se hace desde la rama que recibe los cambios;
- el historial puede dejar de ser una línea única completamente simple;

- Git permite integrar líneas de trabajo separadas.

---

9. Reconocer un conflicto representado

Ejercicio 13. Identificar marcadores de conflicto

Lee este ejemplo de archivo con conflicto:

Archivo de ejemplo: `README.md`

```
<<<<<<< HEAD
# Manual Git Local

Este repositorio contiene apuntes iniciales sobre workspace, index y commits.
=======
# Manual de Git en local

Este repositorio reúne notas introductorias sobre staging, ramas y control de
versiones.
>>>>>>> docs/guia-inicial
```

Tu tarea

Debes ser capaz de identificar:

- dónde empieza el conflicto;
- dónde termina;
- qué dos versiones están compitiendo;
- por qué ese archivo todavía no puede considerarse válido.

No necesitas resolverlo todavía en esta sesión. Basta con reconocer su estructura.

---

10. Mini laboratorio individual

Ejercicio 14. Construir un mini repositorio bien organizado

Crea una nueva carpeta llamada `laboratorio-git-local` y conviértela en un pequeño repositorio documental sobre Git.

Debe incluir al menos:

- `README.md`
- `glosario.md`
- `comandos-basicos.md`
- `preguntas-frecuentes.md`
- `.gitignore`

Debes cumplir estas condiciones obligatorias

1. inicializar correctamente el repositorio;
2. redactar los archivos en Markdown;
3. usar al menos **cinco commits**;
4. usar **Conventional Commits** en todos los mensajes;
5. incluir al menos un `docs:`, un `feat:` y un `chore:`;
6. crear al menos una rama;
7. integrar esa rama en la principal;
8. usar `git commit --amend` una vez de forma justificada;
9. usar `stash` una vez de forma justificada.

Restricciones

- No puedes hacer un único commit grande con todo.
- No puedes usar mensajes ambiguos.
- No debes usar HTML.
- Debes comprobar varias veces el estado y el historial.

> ✓ **Objetivo del laboratorio**
>
> Si este ejercicio te sale bien, ya no solo conoces comandos sueltos: empiezas a trabajar con Git como sistema de organización real.

---

11. Autoevaluación final

Ejercicio 15. Cierre de la sesión

Crea un único archivo llamado `autoevaluacion-sesion-02.md` y responde brevemente:

1. ¿Qué diferencia hay entre workspace, index y repositorio local?
2. ¿Qué papel cumple `stash`?

3. ¿Qué ventaja tiene usar `README.md` y Markdown en este bloque?
4. ¿Qué diferencia hay entre `git status`, `git diff` y `git log`?
5. ¿Qué hace `git commit -am` y cuál es su límite?
6. ¿Para qué sirve `git commit --amend`?
7. ¿Qué aporta usar Conventional Commits?
8. ¿Qué es una rama y por qué te interesa usarla?

Cierre

Al terminar esta hoja deberías ser capaz de trabajar con Git en local de forma razonada, construir un pequeño historial legible, usar Markdown en documentación básica, entender el papel del staging y dar tus primeros pasos con ramas y merges sencillos.

Entregable recomendado

La evidencia principal de esta sesión debe ser:

- el repositorio `manual-git-local`;
- el repositorio `laboratorio-git-local`;
- el archivo `observaciones-lgb.md`;
- el archivo `autoevaluacion-sesion-02.md`.

La fuente principal de evaluación debe ser el propio historial Git del trabajo realizado.

Recursos de apoyo

Documentación oficial y manuales

- Git Book en español: `https://git-scm.com/book/es/v2`
- Documentación oficial de Git: `https://git-scm.com/docs/git/es`

Documentación específica de apoyo

- Conventional Commits: `https://www.conventionalcommits.org/`
- Documentación de GitHub sobre Markdown: `https://docs.github.com/es/get-started/writing-on-github`

Recursos visuales y de consulta

- Learn Git Branching: `https://learngitbranching.js.org/?locale=es_ES`
- Explain Git with D3: `https://onlywei.github.io/explain-git-with-d3/`

- Git Exercises: `https://gitexercises.fracz.com/`